

Architecture note — Media Monitoring Agent

Problem. Six analysts spend 3+ hours each morning reading ~40 outlets to brief the Director General by 07:30. Quality varies by shift, items get missed, and negative stories breaking outside the morning cycle can sit unnoticed for hours.

Solution in one line. A pipeline reads the coverage, tags it, drafts a cited briefing, and hands it to a human to approve. The analysts stay in charge; the machine does the reading.

Components

Component	Role	Why this choice
Python (agent/)	Ingest, de-duplicate, classify, draft, check	Plain functions; every step runs and tests on its own
Claude Haiku 4.5	Classify each item (theme, sentiment, priority, one-line reason)	1,500 small decisions a day: speed and cost dominate
Claude Sonnet 5.5	Write the briefing	One document a day read by a Director General: quality dominates
SQLite	Archive of articles and briefings, inside the client's environment	One file, no server, standard SQL; Postgres later is a small change
FastAPI (api.py)	Nine endpoints so n8n can press each step	Adds no logic of its own; API-key auth; clean JSON errors
Archive Q&A (agent/ask.py)	Plain-language questions over the archive	Keyword retrieval + grounded answer; same citation check as the briefing
n8n	Scheduling, human approval, delivery, alert cadence	The backbone the brief requires; the client's team can read and change the flow without code

LLM choice and alternatives: see `docs/llm-choice.md` .

Data flow

```
06:00  Schedule → POST /pipeline/daily → draft emailed to analyst → WAIT (form)
                                             approved → record → HTML
email to DG office
                                             rejected → record → notify
analyst

every 10 min  Schedule → ingest → classify → pending high-risk? → ONE alert
email → acknowledge
```

How the Director can trust what reaches the DG's desk

1. **Nothing is sent without a named human approving it.** The decision, name and time are stored against the briefing.
2. **Every claim carries a citation**, and code verifies each [n] points to an article that was actually supplied. A made-up citation is flagged at the top of the draft.
3. **The model is told what it cannot do** — only use supplied items, no outside facts — and in practice it declines to connect stories without evidence ("the items do not say whether these are the same incident").
4. **The high-risk rule is written in code, not learned:** negative + high priority + a sector theme. The client can read it and change it.
5. **Measured, not assumed:** 83% theme / 80% sentiment agreement with a human labeller on a blind sample, with the disagreements listed. The boundary cases are exactly why approval is mandatory.
6. **Failures are visible, never silent:** dead feeds, truncated drafts and bad citations appear as warnings on the draft; a failing step stops the n8n workflow rather than emailing nothing.

Security and data residency

- Runs entirely inside the client's environment: n8n and the database are local; the only outbound traffic is article text to the LLM API over HTTPS. Internal documents, when added later, stay in the same database and never leave.
- Secrets live in `.env`, never in code or Git. The API requires a key on every call.
- Production path: HTTPS in front of n8n and the API; role-based access in n8n (analysts approve, DG office reads); a regional LLM endpoint or VPC-hosted model if policy requires no data leaving the Kingdom; audit log = the `briefings` table plus n8n execution history.

Cost (1,500 items/day)

≈ \$33/month in LLM calls (Haiku classification ≈ \$31, Sonnet briefing ≈ \$2), ≈ \$19 with prompt caching, plus a small VM for n8n. Against: 3 hours × 6 analysts every morning.

Scope cuts (deliberate)

Arabic sources and WhatsApp/Teams delivery. The brief says to protect the core loop first; it runs end to end and both approval branches are tested.

Next two weeks

1. Arabic sources (the models already read Arabic; the work is feeds and a bilingual codebook).
2. Story clustering so 16 outlets on one story become one item.
3. Embedding-based retrieval for the archive Q&A, so questions match meaning rather than keywords.
4. A 200-item evaluation set labelled by two client analysts, with agreement reported.
5. Analyst edits captured in the approval form and fed back as examples for the prompt.
6. **Client-chosen LLM provider.** A provider abstraction in `agent/llm.py` selected by an `LLM_PROVIDER` setting (Anthropic, OpenAI, Google, or a Gulf-hosted model), so the client picks the vendor that satisfies their procurement and data-residency rules. The evaluation set is re-run on the client's own data before any switch, so the choice is measured, not assumed.
7. **More delivery channels.** Email today; Microsoft Teams, Slack and WhatsApp Business are each a single n8n node on the same approved-briefing branch, so the DG office and the analysts can each receive the briefing and alerts where they already work. Channel choice becomes a setting, not a code change.